

ARM Assembly Instruction Handbook

Exam-focused Cortex-M3 / Thumb-2 reference derived from the ARM lecture set, all 23 indexed ARM papers, professor assembly templates and historical answer sources.

How to use this during an exam

Start from the operation you need, then check four things before copying a pattern: element width, signedness, flags, and AAPCS preservation. The examples use Keil assembly syntax. Instruction availability and immediate encoding depend on Thumb-2; when an immediate is rejected, load it into a register or use a literal.

AAPCS checkpoint

R0-R3 carry the first four arguments and the return begins in R0. Additional arguments are at the caller's stack. R4-R11 and SP are callee-saved. LR must be saved by any non-leaf routine. Keep SP 8-byte aligned whenever control crosses a public function boundary.

Condition-code selection

Suffix	Meaning	Use
EQ / NE	equal / not equal	zero, sentinel, equality
LO / HS	unsigned < / >=	sizes, indexes, addresses
LS / HI	unsigned <= / >	inclusive unsigned bounds
LT / GE	signed < / >=	signed bytes/words
LE / GT	signed <= / >	signed inclusive bounds
MI / PL	negative / non-negative	sign test
CS / CC	carry set / clear	carry or no-borrow / borrow
VS / VC	overflow set / clear	signed overflow contract

Data movement and constants

MOV, MOVS

Use: Copy a register or encodable immediate. MOVS updates N and Z; use MOV when flags must survive. Large constants may need MOVW/MOVT or LDR =constant.

```
MOV R4, R0; MOVS R6, #0
```

Constraints: MOV cannot encode every 32-bit immediate. Do not use MOVS accidentally between CMP and a conditional branch.

MOVW, MOVT

Use: Build a 32-bit constant without a literal-pool load. MOVW writes the low half; MOVT writes the high half without changing the low half.

```
MOVW R0, #0xCDEF; MOVT R0, #0x89AB
```

Constraints: Use when the assembler supports the Thumb-2 forms. Neither is a memory access.

ADR, ADRL

Use: Form the address of a nearby label using the current PC. ADRL is an assembler expansion for a wider range.

```
ADR R0, table
```

Constraints: Range and alignment are assembler-dependent. LDR R0, =label is more general but uses a literal or relocation.

LDR Rd, =expression

Use: Assembler pseudo-instruction for an address or constant. It becomes MOV/MOVW/MOVT when possible or a PC-relative literal load.

```
LDR R4, =outputData
```

Constraints: Keep LTORG far enough from executed code and within literal-load range. This is not the same as LDR Rd, [Rn].

MRS, MSR

Use: Move from or to special registers such as APSR, CONTROL, MSP, PSP, PRIMASK, CFSR-related system state.

```
MRS R0, APSR; MSR APSR_nzcvq, R1
```

Constraints: Only writable fields may be changed. Privileged registers require privileged execution. Preserve unrelated APSR bits when the paper requires them unchanged.

Loads, stores, arrays and matrices

LDR, STR

Use: Load or store a 32-bit word. Use for `uint32_t/int` arrays and stacked words.

```
LDR R5, [R4, R6, LSL #2]; STR R5, [R0, #8]
```

Constraints: Word index scale is 2. Respect alignment. LDR changes no flags.

LDRB, STRB

Use: Load or store one unsigned byte. LDRB zero-extends into the register.

```
LDRB R3, [R0], #1; STRB R3, [R1], #1
```

Constraints: Use LDRSB, not LDRB, when a signed `int8_t` comparison is required.

LDRSB

Use: Load a signed byte and sign-extend it to 32 bits.

```
LDRSB R7, [R4, R6]
```

Constraints: This is why the insertion-sort answer orders negative bytes correctly.

LDRH, STRH

Use: Load or store a 16-bit halfword. LDRH zero-extends.

```
LDRH R0, [R1, R2, LSL #1]
```

Constraints: Use LDRSH for signed `int16_t` data. Halfword index scale is 1.

LDRD, STRD

Use: Transfer a pair of consecutive words.

```
LDRD R0, R1, [R2]
```

Constraints: Register pairing and address alignment constraints depend on the encoding. Do not assume it updates flags.

Pre/post indexing

Use: Combine a transfer with pointer movement. Pre-index changes the address before access; post-index accesses then changes it.

```
LDRB R3, [R0], #1; STR R2, [R1, #4]!
```

Constraints: Writeback modifies the base register. Avoid unpredictable combinations where the base is also a transferred register.

LDM/STM, LDMIA/STMIA

Use: Transfer multiple registers, useful for blocks and context work.

```
LDMIA R0!, {R2-R5}
```

Constraints: Register order follows register number, not textual order. The base writeback must be intentional.

PUSH, POP

Use: Stack aliases for storing/loading register lists. Save callee-saved registers and LR in non-leaf routines; return with `POP {...,PC}`.

```
PUSH {R4-R7, LR}; ...; POP {R4-R7, PC}
```

Constraints: AAPCS requires SP to be 8-byte aligned at public call boundaries. Pushing an odd count of 32-bit registers misaligns SP unless compensated.

Arithmetic

ADD, ADDS

Use: Add operands. The S form updates N, Z, C and V.

```
ADDS R0, R0, R1
```

Constraints: Use ADDS when the following decision needs flags; use ADD when existing flags must survive.

ADC, ADCS

Use: Add with the current carry flag. Combine with ADDS for multiword addition.

```
ADDS R0, R0, R2; ADCS R1, R1, R3
```

Constraints: Carry-in must be established deliberately. ADC without S preserves flags.

SUB, SUBS

Use: Subtract. SUBS updates flags and is also common for countdown loops.

```
SUBS R2, R2, #1; BNE loop
```

Constraints: After CMP/SUBS, unsigned and signed branches interpret the same flags differently.

SBC, SBCS

Use: Subtract with borrow using inverted carry semantics. Combine with SUBS for multiword subtraction.

```
SUBS R0, R0, R2; SBCS R1, R1, R3
```

Constraints: C=1 means no borrow. This is a frequent source of reversed logic.

RSB, RSBS

Use: Reverse subtraction: operand2 minus operand1. With zero it forms two's-complement negation.

```
RSBS R0, R0, #0
```

Constraints: Negating 0x80000000 overflows in signed arithmetic; inspect V if required.

MUL

Use: Multiply the low 32 bits of two operands.

```
MUL R0, R0, R1
```

Constraints: High overflow is discarded. MUL does not replace UMULL/SMULL when the full 64-bit product is required.

MLA, MLS

Use: Multiply-accumulate or multiply-subtract. MLS is especially useful for remainder after division.

```
UDIV R1, R0, R2; MLS R3, R1, R2, R0
```

Constraints: MLS computes accumulator minus product; verify operand order.

UDIV, SDIV

Use: Unsigned or signed 32-bit division.

```
UDIV R7, R5, R10; SDIV R6, R0, R1
```

Constraints: They return a quotient only and do not set condition flags. Divide-by-zero behavior depends on CCR trapping. Papers may forbid SDIV and require a manual 64/32 algorithm.

UMULL, SMULL, UMLAL, SMLAL

Use: Produce or accumulate a full 64-bit result in a low/high register pair.

```
UMULL R0, R1, R2, R3
```

Constraints: Low and high destinations must be distinct and chosen carefully. Signed and unsigned high halves differ.

Bitwise operations and shifts

AND, ANDS, TST

Use: AND masks bits. ANDS stores the result and sets N/Z; TST sets flags but discards the result.

```
ANDS R0, R0, #0xFF; TST R1, #1
```

Constraints: TST is preferable when only the branch decision matters.

ORR, ORRS

Use: Set or combine bits.

```
ORR R0, R0, R1
```

Constraints: Use BIC to clear known bits rather than ORR with an inverted mask.

EOR, EORS

Use: Exclusive OR, used for parity, toggling, matrix arithmetic and LCG variants.

```
EORS R9, R9, R12
```

Constraints: XORing a bit twice cancels it. The S form overwrites N/Z.

BIC, BICS

Use: Clear selected bits: $Rd = Rn \text{ AND NOT operand2}$.

```
BIC R0, R0, #(3 << 4)
```

Constraints: Useful for read-modify-write register configuration before ORR sets the desired field.

MVN, MVNS

Use: Bitwise NOT of the operand.

```
MVN R0, R0
```

Constraints: Often combined with ADDS #1 for two's complement when RSB is unsuitable.

LSL, LSLs

Use: Logical left shift; fills low bits with zero and can provide an index scale or bit mask.

```
LDR R0, [R4, R6, LSL #2]
```

Constraints: Large shifts and flag behavior differ between immediate and register forms. LSL by 2 scales a word index.

LSR, LSRs

Use: Logical right shift; fills high bits with zero. Use for unsigned fields.

```
LSR R0, R0, #8
```

Constraints: Do not use it to divide a negative signed value; ASR preserves the sign.

ASR, ASRS

Use: Arithmetic right shift; replicates the sign bit.

```
ASR R0, R0, #1
```

Constraints: Rounding for negative values is toward negative infinity, not C signed division's truncation toward zero.

ROR, RRX

Use: Rotate right; RRX rotates through carry by one bit.

```
ROR R0, R0, #8
```

Constraints: RRX consumes and replaces carry, so establish C deliberately.

UXTB, UXTH, SXTB, SXTH

Use: Explicitly zero- or sign-extend an 8- or 16-bit value already in a register.

```
UXTB R5, R1
```

Constraints: Loads may already extend; use these after arithmetic or when documenting a narrow contract.

REV, REV16, REVSH

Use: Reverse byte order in a word, each halfword, or a signed halfword.

```
REV R0, R0
```

Constraints: Use for endianness conversion, not bit reversal.

CLZ

Use: Count leading zero bits.

```
CLZ R0, R1
```

Constraints: Useful for normalization and bit length. Define the desired behavior for input zero.

Comparison and conditional control

CMP, CMN

Use: Set flags for subtraction or addition without storing a result.

```
CMP R6, R5; BHS done
```

Constraints: CMP feeds both signed and unsigned branch families; choose the family that matches the data type.

B

Use: Unconditional branch to a local label.

```
B outer_loop
```

Constraints: It does not save a return address. Use BL for a subroutine call.

BEQ/BNE

Use: Branch when Z is set/not set, typically after CMP, TST or an S-form arithmetic instruction.

```
CMP R0, #0; BEQ empty
```

Constraints: Any intervening flag-setting instruction destroys the comparison result.

BLO/BHS and BCC/BCS

Use: Unsigned lower/higher-or-same; aliases based on carry clear/set.

```
CMP R6, R5; BHS done
```

Constraints: Use for sizes, addresses and uint values. Do not use BLT/BGE for unsigned lengths.

BLS/BHI

Use: Unsigned lower-or-same/higher.

```
CMP R5, #1; BLS done
```

Constraints: These include equality differently from BLO/BHS.

BLT/BGE

Use: Signed less-than/greater-or-equal using N and V.

```
CMP R8, #0; BLT insert
```

Constraints: Required for signed array elements and signed candidates.

BLE/BGT

Use: Signed less-or-equal/greater-than.

```
CMP R0, R7; BLE insert
```

Constraints: Do not substitute unsigned BLS/BHI when negatives are possible.

BMI/BPL

Use: Branch on negative/non-negative according to N.

```
CMP R0, #0; BMI negative
```

Constraints: After arithmetic overflow, N alone may not represent signed relational ordering; CMP plus BLT/BGE is safer.

CBZ/CBNZ

Use: Compare a low register with zero and branch without changing flags.

```
CBZ R0, empty
```

Constraints: Branch range and eligible registers are encoding-dependent. Useful when current flags must survive.

IT/ITE and conditional suffixes

Use: Condition one to four following Thumb instructions. ITE selects then/else forms.

```
CMP R0, #0; ITE EQ; MOVEQ R1,#1; MOVNE R1,#0
```

Constraints: Instructions in the block need matching condition suffixes. Prefer ordinary branches for long or changing logic.

Calls, returns, exceptions and concurrency

BL

Use: Call a subroutine by writing the return address to LR.

```
BL aliquotSum
```

Constraints: A routine that executes BL is non-leaf and must preserve its incoming LR before the first nested call.

BX, BLX

Use: Branch to a register; BX LR is the normal leaf return. BLX also writes LR and may change instruction state.

```
BX LR
```

Constraints: Cortex-M code must remain in Thumb state; function addresses have bit 0 set.

SVC

Use: Enter the supervisor-call exception with an 8-bit immediate service number.

```
SVC #50
```

Constraints: The handler normally decodes the immediate from the halfword at stacked PC minus 2. Select MSP or PSP from EXC_RETURN.

BKPT

Use: Enter the debugger with an immediate breakpoint number.

```
BKPT #0
```

Constraints: Without a debugger, behavior can escalate to a fault. Do not leave test breakpoints in the submitted flow.

CPSID/CPSIE

Use: Disable or enable configurable interrupts, usually with operand i.

```
CPSID i; ...; CPSIE i
```

Constraints: Do not blindly re-enable interrupts if they were already disabled; saving/restoring PRIMASK is safer.

DMB, DSB, ISB

Use: Memory, completion and instruction-stream barriers.

```
DMB; MSR CONTROL, R0; ISB
```

Constraints: Use for synchronization and system-control changes, not as a substitute for volatile or correct ownership.

LDREX/STREX, CLREX

Use: Exclusive load/store pair for lock-free updates. STREX reports whether the reservation succeeded.

```
retry: LDREX R1, [R0]; ADD R1, #1; STREX R2, R1, [R0]; CBNZ R2, retry
```

Constraints: Interrupts or other writes may clear the reservation. Always loop on STREX failure.

WFI, WFE, SEV, NOP

Use: Wait for interrupt/event, send event, or execute no operation.

```
WFI
```

Constraints: WFI is safe only when an enabled event can wake the processor. It does not configure the peripheral or clear pending flags.

Assembly solution recipes

These are complete shapes rather than isolated instructions. Match the paper's prototype, widths, signedness and output contract before adapting one.

Leaf function with four register arguments

Recognition: A short formula or comparison with no nested function call.

```
; uint32_t f(uint32_t a, uint32_t b, uint32_t c, uint32_t d)
f PROC
    ADD    r0, r0, r1
    MLA    r0, r2, r3, r0
    BX     lr
    ENDP
```

Checks: R0-R3 are caller-saved. No stack frame is needed when no callee-saved register or nested BL is used.

Non-leaf function with aligned frame

Recognition: The routine calls a helper and needs values to survive the call.

```
outer PROC
    PUSH    {r4-r6, lr}        ; 16 bytes, still 8-byte aligned
    MOV     r4, r0             ; preserve input across BL
    BL      helper
    ADD     r0, r0, r4
    POP     {r4-r6, pc}
    ENDP
```

Checks: Save incoming LR before the first BL. Restore every saved register on every return path.

Fifth and sixth stacked arguments

Recognition: The C prototype has more than four 32-bit arguments.

```
; uint32_t f(a,b,c,d,e,f)
f PROC
    PUSH    {r4, lr}          ; SP moved by 8 bytes
    LDR     r4, [sp, #8]       ; original [SP] = fifth argument
    LDR     r12, [sp, #12]     ; original [SP,#4] = sixth
    ADD     r0, r0, r4
    ADD     r0, r0, r12
    POP     {r4, pc}
    ENDP
```

Checks: Stacked offsets are measured after accounting for the callee's own push. Record the frame size before writing offsets.

Bounded word-array sum

Recognition: Traverse uint32_t/int32_t words without reading past length.

```
; r0=base, r1=count, returns r0=sum
sum_words PROC
    MOV     r2, r0
    MOVS    r0, #0
    CBZ     r1, sum_done
sum_loop
    LDR     r3, [r2], #4
    ADD     r0, r0, r3
    SUBS    r1, r1, #1
    BNE     sum_loop
sum_done
    BX     lr
    ENDP
```

Checks: Test count before the first load. ADD wraps modulo 2^{32} unless the contract requires overflow handling.

Signed byte minimum

Recognition: The paper gives int8_t elements or negative byte values.

```
; r0=base, r1=count; count must be nonzero
min_s8 PROC
    LDRSB    r2, [r0], #1
    SUBS     r1, r1, #1
min_loop
    CBZ      r1, min_done
    LDRSB    r3, [r0], #1
    CMP      r3, r2
    BGE      min_keep
    MOV      r2, r3
min_keep
    SUBS     r1, r1, #1
    B        min_loop
min_done
    MOV      r0, r2
    BX       lr
ENDP
```

Checks: LDRSB plus BGE makes both the load and comparison signed. Define behavior for an empty input.

Row-major matrix element

Recognition: Compute matrix[row][column] for a flat word matrix.

```
; r0=base, r1=row, r2=column, r3=column_count
matrix_get PROC
    MLA      r1, r1, r3, r2 ; row*columns + column
    LDR      r0, [r0, r1, LSL #2]
    BX       lr
ENDP
```

Checks: The scale is #2 only for four-byte elements. Validate row/column outside this helper if bounds are part of the contract.

Nested search with early exit

Recognition: Find the first equal pair or stop as soon as the required relation is found.

```
; r0=base, r1=count; returns index pair packed, or -1
find_pair PROC
    PUSH     {r4-r7, lr}
    MOV      r4, r0
    MOV      r5, r1
    MOVS     r6, #0
outer
    ADD      r7, r6, #1
inner
    CMP      r7, r5
    BHS      next_outer
    LDR      r0, [r4, r6, LSL #2]
    LDR      r1, [r4, r7, LSL #2]
    CMP      r0, r1
    BEQ      found
    ADD      r7, r7, #1
    B        inner
next_outer
    ADD      r6, r6, #1
    CMP      r6, r5
    BLO      outer
    MVN      r0, #0
    POP      {r4-r7, pc}
found
    ORR      r0, r7, r6, LSL #16
    POP      {r4-r7, pc}
ENDP
```

Checks: Use unsigned bounds for indexes. Every early return must use the same epilogue.

Frequency table for bytes

Recognition: Count occurrences, digits, symbols, or small bounded keys.

```

; r0=input, r1=count, r2=256-word frequency table (zeroed)
count_bytes PROC
    CBZ     r1, freq_done
freq_loop
    LDRB    r3, [r0], #1
    LDR     r12, [r2, r3, LSL #2]
    ADD     r12, r12, #1
    STR     r12, [r2, r3, LSL #2]
    SUBS    r1, r1, #1
    BNE     freq_loop
freq_done
    BX      lr
ENDP

```

Checks: The output table must have 256 words and must be cleared unless accumulation is required.

Insertion sort signed bytes

Recognition: Small in-place signed array sorting.

```

; r0=base, r1=count
sort_s8 PROC
    PUSH    {r4-r7, lr}
    MOV     r4, r0
    MOVS    r5, #1
sort_outer
    CMP     r5, r1
    BHS     sort_done
    LDRSB   r6, [r4, r5]
    MOV     r7, r5
sort_inner
    CBZ     r7, sort_place
    SUB     r2, r7, #1
    LDRSB   r3, [r4, r2]
    CMP     r3, r6
    BLE     sort_place
    STRB    r3, [r4, r7]
    MOV     r7, r2
    B       sort_inner
sort_place
    STRB    r6, [r4, r7]
    ADD     r5, r5, #1
    B       sort_outer
sort_done
    POP     {r4-r7, pc}
ENDP

```

Checks: Signed load and signed branch must agree. Count 0 and 1 should perform no data access beyond the array.

Recurrence with a policy helper

Recognition: Generate a sequence where only the recurrence formula is likely to change.

```

; r0=output, r1=count
sequence PROC
    PUSH    {r4-r6, lr}
    MOV     r4, r0
    MOV     r5, r1
    MOVS    r6, #0
seq_loop
    CMP     r6, r5
    BHS     seq_done
    MOV     r0, r6
    BL      recurrence_rule
    STR     r0, [r4, r6, LSL #2]
    ADD     r6, r6, #1
    B       seq_loop
seq_done
    MOV     r0, r4
    POP     {r4-r6, pc}
ENDP

```

Checks: Keep the output base and loop state in callee-saved registers because BL may destroy R0-R3 and LR.

Unsigned division and remainder

Recognition: Split decimal digits, calculate modulo, or test divisibility.

```
; r0=value, r1=divisor; returns quotient r0, remainder r1
divmod_u32 PROC
    UDIV    r2, r0, r1
    MLS     r1, r2, r1, r0 ; value - quotient*divisor
    MOV     r0, r2
    BX      lr
ENDP
```

Checks: Check divisor zero before UDIV when trapping or defined error behavior matters.

64-bit addition and subtraction

Recognition: Operate on low/high word pairs.

```
; a=r1:r0, b=r3:r2, returns r1:r0
add_u64 PROC
    ADDS    r0, r0, r2
    ADC     r1, r1, r3
    BX      lr
ENDP
sub_u64 PROC
    SUBS    r0, r0, r2
    SBC     r1, r1, r3
    BX      lr
ENDP
```

Checks: The low-word S instruction establishes carry/no-borrow for the high word.

Return condition flags deliberately

Recognition: The paper grades APSR flags as well as register output.

```
; perform all bookkeeping before the final flag-setting instruction
compare_return PROC
    CMP     r0, r1
    BX      lr ; BX does not change flags
ENDP
```

Checks: No flag-setting instruction may appear between the required final comparison/arithmetic and return.

SVC immediate and MSP/PSP frame selection

Recognition: Decode a supervisor service from the faulting instruction.

```
SVC_Handler PROC
    TST     lr, #4
    ITE     EQ
    MRSEQ   r0, MSP
    MRSNE   r0, PSP
    LDR     r1, [r0, #24] ; stacked PC
    LDRB    r1, [r1, #-2] ; SVC immediate
    B       svc_dispatch_asm
ENDP
```

Checks: The basic exception frame is r0,r1,r2,r3,r12,lr,pc,xPSR. Account for extended frames only if the target uses them.

Atomic update with LDREX/STREX

Recognition: Update shared memory without disabling interrupts for the whole operation.

```
atomic_inc PROC
retry
    LDREX   r1, [r0]
    ADD     r1, r1, #1
    STREX   r2, r1, [r0]
    CBNZ    r2, retry
    DMB
```

```

MOV    r0, r1
BX     lr
ENDP

```

Checks: STREX success is zero. A retry is mandatory because an interrupt or competing write can clear the exclusive reservation.

C and peripheral solution recipes

The high-level API is appropriate when a peripheral is a tool. Use the direct-register variants when configuration itself is graded.

C calls an assembly routine

Recognition: Use an exact prototype shared with the assembly EXPORT.

```

extern uint32_t count_matches(const uint8_t *data,
                             uint32_t count,
                             uint8_t key);

uint32_t answer = count_matches(values, value_count, target);

```

Checks: Pointer element type, signedness and return type must match what the assembly actually loads and returns.

Deferred event state machine

Recognition: Keep callbacks short and perform decisions in the foreground.

```

enum { EVENT_TICK = 1u << 0, EVENT_PRESS = 1u << 1 };

static void timer_cb(uint8_t timer, uint32_t flags)
{
    if (timer == 0u && exam_timer_match_happened(flags, 0u))
        exam_events_set(EVENT_TICK);
}

void exam_user_loop(void)
{
    uint32_t events = exam_events_take(EVENT_TICK | EVENT_PRESS);
    if (events & EVENT_PRESS) { /* transition state */ }
    if (events & EVENT_TICK) { /* advance timed output */ }
}

```

Checks: A bit records that an event occurred, not necessarily how many times. Use a counter when every occurrence must be preserved.

Automatic startup flags

Recognition: Select simple resource startup without adding visible initialization calls to the answer.

```

/* exam_config.h */
#define EXAM_AUTO_START_BUTTONS 1
#define EXAM_AUTO_START_JOYSTICK 1
#define EXAM_AUTO_START_TIMER0 1
#define EXAM_AUTO_START_ADC 1
#define EXAM_AUTO_START_DAC 1

/* exam_user.c */
void exam_user_init(void)
{
    if (exam_auto_init_failures != 0u) {
        /* visible debug breakpoint location */
    }
    /* exam_auto_init_started contains successful AUTO_INIT_* bits. */
}

```

Checks: Automatic timers are free-running. Periodic match values remain question-specific and should use the timer API or direct registers.

All supported devices concurrently

Recognition: Start independent resources together while retaining one owner per vector.

```
#define EXAM_AUTO_START_BUTTONS 1
#define EXAM_AUTO_START_JOYSTICK 1
#define EXAM_AUTO_START_TIMER0 1
#define EXAM_AUTO_START_TIMER1 1
#define EXAM_AUTO_START_TIMER2 1
#define EXAM_AUTO_START_TIMER3 1
#define EXAM_AUTO_START_RIT 1
#define EXAM_AUTO_START_SYSTICK 1
#define EXAM_AUTO_START_ADC 1
#define EXAM_AUTO_START_DAC 1
```

Checks: Buttons and joystick intentionally share scheduler-mode RIT. Raw RIT cannot coexist with that scheduler. DAC table playback later claims one timer, so stop/reassign that timer first.

Periodic timer callback

Recognition: A paper gives a period or rate and accepts a helper-based setup.

```
static void tick(uint8_t timer, uint32_t flags)
{
    if (timer == 1u && exam_timer_match_happened(flags, 0u))
        exam_events_set(1u);
}

void exam_user_init(void)
{
    exam_status_t status = exam_timer_every_ms(1, 500, tick);
    (void)status;
}
```

Checks: The helper owns MR0 and starts the timer. Do not auto-start the same timer first.

Direct Timer register configuration

Recognition: The paper explicitly asks for PR, MR, MCR, TCR or IR values.

```
void timer0_periodic(uint32_t match)
{
    LPC_SC->PCONP |= 1u << 1;
    LPC_TIMER0->TCR = 2u;
    LPC_TIMER0->PR = 0u;
    LPC_TIMER0->MR0 = match;
    LPC_TIMER0->MCR = (1u << 0) | (1u << 1);
    LPC_TIMER0->IR = 0x3Fu;
    NVIC_ClearPendingIRQ(TIMER0_IRQn);
    NVIC_EnableIRQ(TIMER0_IRQn);
    LPC_TIMER0->TCR = 1u;
}
```

Checks: Calculate match from the real PCLK. IR is write-one-to-clear. If defining TIMER0_IRQHandler, transfer vector ownership in exam_config.h.

Free-running elapsed timer

Recognition: Measure time without reset-on-match interrupts.

```
void exam_user_init(void)
{
    (void)exam_timer_clock_divider(2u, 4u);
    (void)exam_timer_prescaler(2u, 24u);
    (void)exam_timer_reset(2u);
    (void)exam_timer_start(2u);
}

uint32_t elapsed_ticks(void)
{
    return exam_timer_count(2u);
}
```

Checks: With 100 MHz core, divider 4 and PR 24, TC advances at 1 MHz. Confirm the course clock configuration before relying on that number.

Debounced buttons

Recognition: INT0/KEY input should create one logical press despite bounce.

```
static void button_cb(exam_button_t button, exam_button_event_t event)
{
    if (button == EXAM_BUTTON_INT0 && event == EXAM_PRESS)
        exam_events_set(1u);
}

void exam_user_init(void)
{
    (void)exam_buttons_start(button_cb);
}
```

Checks: The helper starts scheduler-mode RIT. A direct EINT handler and the callback owner must not own the same vector.

Joystick direction masks

Recognition: Accept single or diagonal joystick input.

```
static void joystick_cb(uint32_t current, uint32_t changed)
{
    uint32_t new_presses = current & changed;
    if (new_presses & EXAM_JOY_UP) exam_events_set(1u << 0);
    if (new_presses & EXAM_JOY_RIGHT) exam_events_set(1u << 1);
}
```

Checks: The values are masks, not mutually exclusive enum alternatives. Test with bitwise AND.

ADC conversion stream

Recognition: Read the LandTiger potentiometer repeatedly.

```
void exam_user_init(void)
{
    (void)exam_pot_start();
}

void exam_user_loop(void)
{
    int sample;
    if (exam_pot_read(&sample) == EXAM_OK) {
        uint8_t level = (uint8_t)((sample * 255L + 2047L) / 4095L);
        (void)exam_led_write(level);
    }
}
```

Checks: The API returns EXAM_NOT_READY until a fresh conversion completes and then automatically starts the next conversion.

DAC lookup-table waveform

Recognition: Output sine/cosine/custom samples at a fixed sample rate.

```
static const uint16_t wave[] = {512, 724, 874, 936, 874, 724, 512,
                                300, 150, 88, 150, 300};

void exam_user_init(void)
{
    (void)dac_play_samples(wave,
        sizeof wave / sizeof wave[0], 12000u, 3u);
}
```

Checks: Every sample must be 0..1023. Playback claims the chosen timer; it cannot share that timer with another running use.

Critical snapshot of shared state

Recognition: Read or modify a multi-field IRQ-shared object consistently.

```
uint32_t saved = exam_critical_enter();
local_count = shared_count;
local_state = shared_state;
exam_critical_exit(saved);
```

Checks: Pass the saved PRIMASK back unchanged. Volatile provides visibility but does not make a multi-step update atomic.

Direct active-low GPIO input

Recognition: The paper grades direct GPIO interpretation.

```
uint32_t pressed;
LPC_GPIO2->FIODIR &= ~(1u << 10);
pressed = ((LPC_GPIO2->FIOPIN & (1u << 10)) == 0u);
```

Checks: Confirm the exact board pin in the paper/schematic. Active-low means a zero electrical level represents pressed.

SVC service in C

Recognition: Dispatch services after the assembly wrapper selects and decodes the exception frame.

```
void svc_dispatch(uint8_t service, svc_context_t *frame)
{
    switch (service) {
        case 1u: frame->r0 = frame->r0 + frame->r1; break;
        case 2u: frame->r0 = exam_led_read(); break;
        default: frame->r0 = 0xFFFFFFFFu; break;
    }
}
```

Checks: Only modify stacked fields intentionally. The service number comes from the SVC instruction, not stacked R0.

Assembler directives that appear in the material

AREA

Declare a code or data section and its attributes.

THUMB

Assemble Thumb instructions.

PRESERVE8

State that code preserves 8-byte stack alignment.

EXPORT / IMPORT

Publish or reference linker symbols.

PROC / ENDP

Mark procedure boundaries for the assembler/debugger.

DCD / DCB

Define word or byte constants.

SPACE

Reserve uninitialized bytes; multiply word counts by four.

EQU / RN

Define a constant or register alias.

ALIGN

Align the following location.

LTORG

Emit the current literal pool.

END

End the assembly source.

High-frequency combinations

Bounded byte loop: LDRB/STRB with post-indexing, SUBS counter, BNE. **Signed sort:** LDRSB, CMP, BLE/BLT.

Word array: LDR/STR with index LSL #2. **Non-leaf:** aligned PUSH, BL, matching POP to PC. **Remainder:** UDIV then MLS. **Peripheral field:** LDR register, BIC field mask, ORR desired value, STR back. **Event handoff:** minimal handler, clear W1C flag, update volatile state, foreground processing.